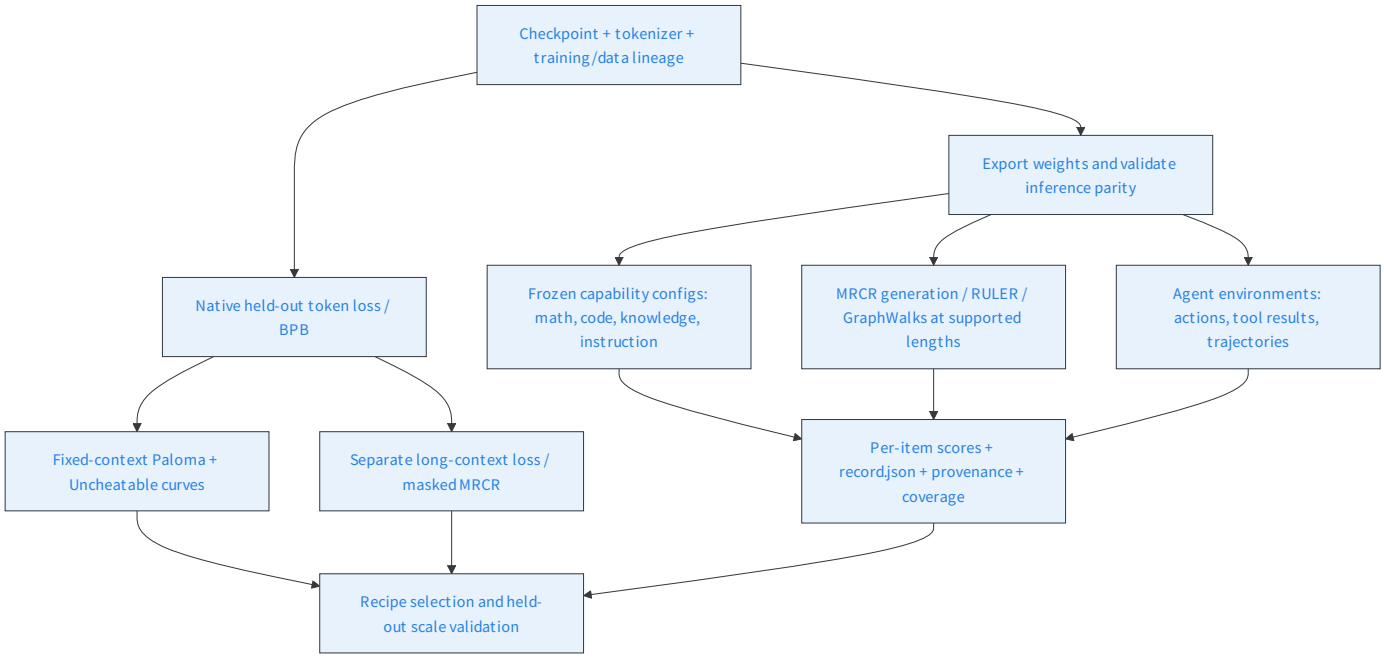
Marin evaluation: pretraining, mid-training and post-training

Exact code paths, datasets, metrics, result records and replication checks

Several evaluation systems, with different purposes

Marin's **in-training held-out loss**, **mixture-search benchmark log-probabilities**, **exported-model capability evaluation** and **agent/reward evaluation** are different measurement pipelines. A Paloma improvement does not establish stronger instruction following, long-context retrieval or cyber task completion. This guide records what the code implements and what the collected results establish. The general evaluator design guide contains our proposed project controls; this page is the Marin-specific account.

Baseline code is pinned to `1d274b52c636b6f2a9177b1d4c14b2f74795a8a6`. Additional PR heads are listed in the saved PR ledger. Model cards/configs, public mixture records and selected code are stored in the evidence bundle. Retrieved PR merge status and published numbers are dated **2026-11-04**; no evaluations were run on GPUs here.



1. Pretraining: build fixed held-out language-model datasets

Paloma

`experiments/datasets/paloma.py` builds validation handles for **16 Paloma subsets**, including web, code, academic and other domains. It reads the `val` split of the registered raw snapshot `raw/paloma-fc6827/65cd6fc`; the underlying `allenai/paloma` dataset has access requirements. The default cache uses the Llama-3 tokenizer.

A concrete reproducibility trap is documented in the function: cache identity is based on **name and version**, not automatically on the supplied tokenizer. A different tokenizer requires a distinct `tag`; otherwise the request can resolve to the pre-existing Llama-3 token cache. The optional detokenized raw reconstruction also requires a fresh version. Save raw-byte hashes, subset mapping, tokenizer repository/revision, packing/context length and scored-token counts. A familiar dataset name alone does not define an evaluation.

Uncheatable

`experiments/datasets/uncheatable.py` adds seven English/code subsets: English Wikipedia, Python and C++ GitHub, BBC news, arXiv physics and computer science, and English AO3. The source points to `ziqing-huang/uncheatable_eval` on mutable `master`; a local version label does not freeze that upstream branch. For a replication, preserve the downloaded archive's hash or commit before processing it. These seven subsets and Paloma's sixteen are not the same universe as the 54-task macro in the historical HF export.

Decontamination joins the training-data DAG

Build evaluation text/features **before** marking the training corpus. PR #8327 describes exact-copy fixtures and benchmark record/feature checks; PR #8328 freezes the historical outputs. The later PR #10541 proposes additional required benchmark coverage but is unmerged in our snapshot. Use the data-lineage guide to connect evaluator versions to contamination-mark versions.

2. Pretraining: checkpoint cadence and metrics

The MoE ladder launcher configures the hero for evaluations every **3,000 steps**, permanent checkpoints every **6,000**, first-step evaluation, full validation (`max_eval_batches=None`), BPB computation and **dropless evaluation**. Smaller rungs choose approximately 20 evaluation points from their own step horizon. These are this launcher's settings; they are not a universal cadence for all historical Marin runs.

Dropless evaluation prevents a capacity-limited expert dispatcher from silently dropping evaluation tokens. Compare like routing modes and record the mesh/capacity settings: training throughput with dropped tokens and dropless held-out loss are distinct quantities. Preserve per-subset totals rather than only the dashboard aggregate.

For scored token positions i, mask m_i, probability p_i and scored UTF-8 byte count n_b, the quantities we need to reproduce are

$$\mathrm{NLL}_{tok} = \frac{-\sum_i m_i \ln p_i}{\sum_i m_i}, \qquad \mathrm{BPB} = \frac{-\sum_i m_i \ln p_i}{n_b \ln 2}.$$

Use the implementation's exact mask/byte accounting, including its EOS/window convention. Token NLL depends on the tokenizer. BPB is useful across tokenizers only when underlying text and scored coverage agree. A macro average gives each declared task/subset a weight; it is not necessarily pooled corpus BPB. Keep router regularization out of the next-token likelihood being compared.

Concrete result: issue #9126 and its completion comment report d1536 final loss **2.54317**, macro BPB **0.91844**, and 15/16 Paloma improvements against the older mixture. The 1.20× number is a scaling-curve compute equivalent assuming equal throughput. The issue explicitly lacks an older Uncheatable baseline, so it does not fit an Uncheatable speedup curve. This is why an evaluation manifest must precede the ladder.

3. Mixture-search evaluation is another protocol

The public 116-row Harrier export stores per-task evaluations, grouped BPB, task counts, target/guardrail aggregates and objective fields. Our saved audit records the schema and an example 54-task result. Its macro BPB around 1.93 is not directly comparable with the 16-subset Paloma macro around 0.92 above.

The newer transfer campaign explicitly selects 20 response-task names, including HumanEval/GSM8K **log-probability** tasks, ARC, OpenBookQA, MMLU-Pro, BBH and other knowledge/reasoning tasks. A `logprob_gsm8k_5shot` metric is not generated-answer GSM8K accuracy. PR #8633's objective code constructs a negative variance-normalized hinge objective over BPB metrics; it does not optimize a simple unweighted accuracy average.

For a record replay, first verify task lists, reference swarm, variance normalization, missing-task behavior and phase/exposure fields. For new proxies, require the same evaluator/task versions before adding observations to that campaign. An optimizer can exploit a changing evaluation definition as readily as it can exploit a useful data mixture.

4. Mid-training: separate context comparability from long-context ability

Fixed-context and longer-context language-model loss

PR #9639, unmerged, keeps **canonical hero validation at 4K** while proposing a 4K→16K training-context transition. That preserves a comparable curve across the handoff. It also makes first-step evaluation occur after resume and preserves cumulative compute accounting. A lower loss at a new context could reflect more conditioning, so do not splice it into the original series without a label.

PR #10711, unmerged, proposes explicit `eval_dropless/seq_len_4096/...` plus current-context and reached 65K/262K contexts. It masks padding, including transitions from real tokens into padding, and reuses device groups. The PR describes **CPU verification, not a completed GPU rehearsal**. Treat this as a concrete implementation to qualify before adopting it.

MRCR likelihood: paired conditioning experiment

PR #7203, unmerged, provides saved transformation code for OpenAI MRCR. It constructs paired `full_context` and `final_user_only` records for 2/4/8 needles, retaining the same reference final answer. The chat generation mask scores the final assistant answer; cache packing uses a right-slicing policy. This asks whether the preceding conversation makes the correct answer more probable. It does **not** ask whether the model generates that answer unaided. Record actual retained context, because slicing can change what the model sees.

MRCR generation: retrieval and answer fidelity

PR #8706, unmerged, adds explicit generation scoring. The code:

1. Bins examples by official `o200k_base` prompt-plus-answer content lengths from 8,192 through 262,144, and 2/4/8 needles.
2. Separately checks **served-model tokenizer** prompt length plus generation budget against its actual context window.
3. Interleaves cells so a limited evaluation does not use only short/easy examples; smoke mode chooses one example per supported cell.
4. Requires the response to start with the supplied nonce; otherwise score is zero. For a prefix hit, strip the nonce and compute `SequenceMatcher` similarity to the reference answer.
5. Reports aggregate similarity, prefix-hit rate and per-length/per-needle metrics.

The baseline MRCR YAML is a separate protocol: 0-shot, batch 1, seed 42, chat template enabled, thinking disabled, 4,096 generation tokens and `max_length=73728`. Do not call that file a 262K benchmark, or assume its runtime equals an unmerged PR head.

RULER and GraphWalks

PR #7626, unmerged, defines 13 RULER tasks plus a smoke suite, propagates maximum sequence lengths and avoids silent left truncation. It describes runtime/dependency work still needed; the PR alone supplies no benchmark score. PR #9511, merged, adds GraphWalks support. Configuration availability is not evidence that every released Marin model passed these suites.

For the actual length-skew treatment design and launch outcomes, use the mid-training guide. Publish sample counts/coverage at each length; do not average unsupported cells into a purported full-window result.

5. Capability evaluation: build the executable evaluator

The baseline evaluation README is the concrete entry point. Suites include `smoke`, `core`, `nlp`, `chat`, `math`, `code` and `agentic`; evals.py maps suites to files. Evalchemy and Harbor run from isolated vendored projects with frozen lockfiles. PR #9813, merged, adds per-evaluation pipeline records; PR #10288, merged, vendors Evalchemy. Historical external evaluator commits require their own recorded checkout.

The source documents these inspection commands, which do not submit evaluation jobs:

```
uv run python -m experiments.evaluation.cli launch \
  --model qwen3-0.6b --evals smoke --dry-run
uv run --project vendor/evalchemy --frozen --no-default-groups evalchemy validate-config \
  experiments/evaluation/configs/evalchemy/gsm8k-smoke.yaml
```

A launch dry run checks configuration/plan shape; runtime task validation checks whether the task exists. Neither proves model inference works. After those checks, qualify a tiny actual evaluation in the intended environment before scheduling a benchmark suite. These commands were inspected, not executed in this repository.

For every benchmark, freeze model weights/revision, tokenizer, chat template/thinking mode, dataset revision/split, few-shot seed/examples, prompt, extractor, max context/generation lengths, sampling, task/runtime commit and scoring implementation. For example, the baseline `gsm8k.yaml` is **5-shot generation**, whereas a separate 0-shot config exists. Its inherited generation settings must be resolved into the result record rather than guessed from the task name.

6. Public HF results: useful anchors, bounded claims

We downloaded cards/configs/API metadata at the following exact revisions, **without model weights**:

Repository/revision	Concrete published evidence	Limits
Marin 8B Base, `e1a2021…`	Base-results table: MMLU 5-shot **67.6**, MMLU 0-shot **65.9**, GSM8K **61.3**	Card describes LM Eval defaults; recovering the exact historical harness/task commits remains necessary
Marin 8B Instruct, `0378f9c…`	Public instruct model metadata/card	The card also contains base-model benchmark tables; do not relabel those rows as instruct scores
Marin 32B Base, `f19b346…`	Mantis: HumanEval **42.7**, GSM8K **69.1**, MATH **15.3**; Bison: **29.3/54.7/10.4**	Author-reported base-model percentages, not our reruns or an SFT comparison
Delphi endpoint, `56cc5aa…`	Concrete HF initialization artifact used by the SFT config	`32p07b` in its name denotes the token horizon; do not infer 32B parameters
Delphi Magpie cold-start, `8be0758…`	Concrete post-training initialization reference in the RL-probe family	Repository availability does not establish the outcome of the unmerged probe launcher

The Marin-32B retrospective identifies contamination in Bison's Dolmino-derived data and the subsequent Mantis correction. Consequently the Bison→Mantis table is not a clean single-variable causal estimate. It also describes that release as a base model without SFT/RLHF or long-context extension. Keep it distinct from later Grug/Snowball MoE experiments.

7. Post-training: evaluate behavior and generalization

SFT preprocessing must preserve assistant-only masks, canonical conversation boundaries and the deployed template. Export changes can alter usable behavior: PR #7280, merged, prepares reserved think/tool token rows rather than merely editing strings. Validate reference-token logits/native-versus-exported loss and prompt formatting before comparing generated-answer scores. PR #10707, unmerged, specifically addresses export verification against trained master weights; it is a proposed implementation, not evidence all exports already pass.

For RL or synthetic curricula, report **base/control/treatment**, equal training budgets, per-item attempts and uncertainty. PR #10623, unmerged, supplies a concrete RSI protocol: four attempts per evaluation task, paired bootstrap intervals, held-out pool tasks and synthesized-family probes. Its Snowball 67B-A2B example improves synthesized-family probes from control **0.549** to treatment **0.820**, but falls from **0.499** to **0.448** on 300 Smartie validation tasks. These are reported mean scores under that protocol, not a new broad capability win. Seed-0 intervals for IFEval, IFBench, GSM8K, MATH500, HumanEval+ and MBPP+ include zero. See the saved experiment README.

For cyber agents, Harbor supports task environments and trajectory capture. Freeze one agent configuration and one versioned task dataset per evaluation. The baseline `swebench.yaml` uses Terminus-2/Daytona with a dataset reference `main`: replace mutable refs with a specific dataset commit for replication. Record tool/image versions, timeouts, attempts, environment failures and trajectories. Existing config support does not mean Marin has measured our proposed cyber use case.

8. Publish the full evaluation receipt

The launcher writes `{records_prefix}/{run_id}/record.json`, including resolved model/evaluator settings, runtime provenance, hardware, status, metrics, coverage, result path and Iris job IDs. Runs sharing a server retain a group ID. FineStore archives include `samples` (per-item scores and trial IDs), `evalchemy_results`, `steps` and `rollouts_v1`; Harbor also retains native trial results. Baseline result contract.

Coverage is scored items / attempted items; infrastructure failures do not count as scores. Runs with infrastructure errors and coverage below **90%** become `infra_failed` and are not cached as successful. Concurrent evaluations omit server-wide throughput attribution because shared counters cannot be assigned reliably to individual evaluators. Preserve these statuses when exporting a results table.

Our team should archive this minimum receipt for every selected checkpoint:

Artifact	Required content
Checkpoint lineage	Training checkpoint and HF revision; stored/active parameters; consumed tokens; optimizer state/reset; context; data mixture revision
Evaluation manifest	Dataset/checksum, task/runtime commit, prompts/masks, shot/seed settings, tokenizer/template, decoding and limits
Native validation	Per-subset loss sums/token/byte counts, macro definition, routing mode, length, scored coverage
Capability results	Per-item predictions/scores, extraction failures, attempts and paired confidence intervals
Agent results	Environment/image/tool versions, trajectories, verifier outputs, timeouts and infra failures
Final record	Status, hardware, coverage, paths/hashes and comparison cohort

This schema extends Marin's concrete record contract for our cross-trainer work. It is a proposed handoff requirement, not a claim all historical public model cards provide every field.